

n8n Telegram Personal Assistant Workflow Guide

This guide explains how the active n8n workflow is built and how to maintain it. It is written for someone who wants to understand the project well enough to explain it, debug it, or build a similar assistant later.

Active workflow:

- Name: Telegram Personal Assistant - Inbox Budget Commands
- n8n workflow ID: telegram-personal-assistant-inbox-budget-commands
- Main export files:
 - workflows/current-inbox-after-budget-commands.json - workflows/telegram-assistant-inbox-budget-commands.json

1. What This Workflow Does

The workflow turns Telegram messages into structured personal-assistant actions.

It can handle:

- Normal text expenses, for example `spent 25 aed at Costa with ENBD Visa`
- Uncertain expenses, for example `maybe spent around 20 at Costa`
- Receipt photos
- Voice notes
- Tasks and reminders
- Task completion, for example `done buy printer ink`
- Memory commands, for example `remember Costa is Food`
- Budget commands, for example `groceries 2k`, `list budgets`
- Pending confirmations using `confirm` / `cancel`
- Undo last expense
- Budget warnings after saving expenses

The core idea is simple:

```
Telegram message
-> normalize the input
-> ignore duplicate Telegram retries
-> route special commands first
-> if needed, ask OpenAI to parse the message
-> save or update data in Supabase
-> reply back in Telegram
```

2. Main Systems Involved

The project is made from four main services.

Telegram

Telegram is the user interface. The user sends text, voice, photos, and button callbacks to the bot.

Important workflow nodes:

- Telegram Inbox
- Telegram reply nodes such as Confirm Expense , Confirm Budget Command , Confirm Unknown

n8n

n8n is the automation engine. It receives the Telegram webhook, runs the workflow nodes, calls APIs, and sends Telegram replies.

n8n stores the workflow locally, and this repo keeps exported JSON copies so the workflow can be versioned in Git.

OpenAI

OpenAI is used for:

- Parsing free-form text into JSON
- Reading receipt photos
- Transcribing voice notes
- Creating memory embeddings

Important workflow nodes:

- OpenAI Parse Message
- Transcribe Voice
- Create Memory Embedding

Supabase

Supabase stores the assistant data.

It stores things like:

- Expenses
- Tasks
- Reminders
- Pending actions
- Memories
- Budgets
- Processed Telegram update IDs

The workflow talks to Supabase through REST endpoints and RPC-style tables/views.

3. High-Level Workflow Map

Here is the workflow in plain English:

```
Telegram Inbox
-> Normalize Telegram Input
-> Check Telegram Update Dedupe
    -> duplicate? Reply Duplicate Telegram Update
    -> new update? Mark Telegram Update Processed
        -> Route Pending Command
        -> Prepare Callback Ack
```

```
Route Pending Command
-> confirm/cancel/undo/memory/budget commands
-> otherwise Route Voice
```

```
Route Voice
-> voice note path
-> photo path
-> text path
```

```
Text/photo/voice after preparation
-> Read Parser Context
-> Read Recent Memories
-> Apply Memory Context
-> OpenAI Parse Message
-> Parse OpenAI Result
-> Route Intent
-> save/update in Supabase
-> Telegram confirmation
```

The important design choice is that deterministic commands are handled before OpenAI. For example, `list budgets` does not need AI parsing, so the workflow routes it directly to the budget-list branch.

4. Step-by-Step Node Explanation

Step 1: Telegram Receives the Message

Node: Telegram Inbox

This is the webhook entry point. Telegram sends updates here when the bot receives:

- Text messages
- Voice notes
- Receipt photos
- Callback queries from buttons

This node also downloads binary data for media, such as voice files and photos.

Step 2: Normalize Telegram Input

Node: Normalize Telegram Input

This is one of the most important Code nodes. It converts Telegram's raw update shape into one consistent internal shape.

It extracts:

- `chatId`
- `telegramUpdateId`
- Message text or caption
- Callback data
- Whether the message is text, voice, or photo
- Binary key for media files
- Budget commands
- Memory commands
- Pending commands like `confirm`, `cancel`, and `undo`

- The first draft of the OpenAI request body

Why this matters:

Telegram messages, photos, voice notes, and callback queries all arrive in slightly different JSON shapes. Normalization gives the rest of the workflow a predictable object to work with.

Step 3: Telegram Update Deduplication

Nodes:

- Check Telegram Update Dedupe
- Route Telegram Update Dedupe
- Mark Telegram Update Processed
- Reply Duplicate Telegram Update

Telegram may retry webhook delivery. Without dedupe, one message could create duplicate expenses or tasks.

The workflow stores every `telegramUpdateId` in Supabase:

```
assistant.processed_telegram_updates  
public.assistant_processed_telegram_updates
```

If the update ID already exists, the workflow replies:

```
I already processed that Telegram update.
```

If it is new, processing continues.

Step 4: Callback Acknowledgement

Nodes:

- Prepare Callback Ack
- Answer Callback

When the user taps an inline button in Telegram, Telegram expects the bot to acknowledge the callback. This branch does that, so Telegram does not keep showing a loading spinner.

This runs alongside the real confirm/cancel handling.

Step 5: Route Pending and Direct Commands

Node: Route Pending Command

This switch sends command-style messages down direct paths before using OpenAI.

Examples:

- `confirm` -> confirmation path
- `cancel` -> cancellation path
- `undo` -> undo last expense path
- `remember ...` -> memory creation path
- `forget ...` -> memory delete path
- `show memories` -> memory recall path
- `groceries 2k` -> budget upsert path

- `list budgets` -> budget list path
- no command -> continue to text/photo/voice parsing

This makes the workflow faster and safer because clear commands are handled by normal code, not AI guessing.

5. Text Expense Path

Example:

```
spent 25 aed at Costa with ENBD Visa
```

Flow:

```
Route Pending Command
-> Route Voice
-> Read Parser Context
-> Read Recent Memories
-> Apply Memory Context
-> OpenAI Parse Message
-> Parse OpenAI Result
-> Route Intent
-> Prepare Expense Row
-> Append Expense
-> Read Budget After Expense
-> Read Monthly Spend After Expense
-> Append Budget Warning
-> Confirm Expense
```

What happens:

1. OpenAI parses the text into structured JSON. 2. The workflow validates the result. 3. The expense row is prepared. 4. The expense is inserted into Supabase. 5. The current monthly category spend is checked. 6. If the user is near or over budget, the Telegram confirmation includes a warning.

6. Uncertain Expense Path

Example:

```
maybe spent around 20 at Costa
```

If OpenAI returns low confidence or missing fields, the workflow does not immediately save the expense.

Flow:

```
Parse OpenAI Result
-> Route Intent
-> Route Clarify or Pending
  -> Confirm Unknown
  -> Prepare Pending Action
  -> Append Pending Action
  -> Confirm Pending Request
```

The pending action is saved in Supabase. The user can later reply:

confirm

or:

cancel

Safety rule:

- Button callbacks can target an exact pending action ID.
- Text `confirm` / `cancel` only targets the latest pending action for the same chat ID that is still pending and not expired.

This prevents confirming an old or unrelated action.

7. Receipt Photo Path

Example:

The user sends a receipt image to Telegram.

Flow:

```
Telegram Inbox
-> Normalize Telegram Input
-> Check Telegram Update Dedupe
-> Route Pending Command
-> Route Voice
-> Prepare Photo Image
-> Read Parser Context
-> Read Recent Memories
-> Apply Memory Context
-> OpenAI Parse Message
-> Parse OpenAI Result
-> Route Intent
```

Important node: `Prepare Photo Image`

n8n stores downloaded Telegram photos as binary files. OpenAI needs the image content as base64 or a valid image URL.

`Prepare Photo Image` does this:

```
n8n binary file
-> read file buffer
-> convert to base64
-> attach as data:image/jpeg;base64,...
-> send to OpenAI
```

Important maintenance lesson:

The photo path recently broke because a later node rebuilt the OpenAI body from the older normalized payload and accidentally restored `filesystem-v2` instead of the real base64 image. The fix was to make `Apply Memory Context` start from `Prepare Photo Image` for photo messages.

That is a good example of workflow debugging:

```
Find where the data is correct
-> find where it becomes wrong
-> fix the first node that mutates it incorrectly
```

8. Voice Note Path

Flow:

```
Route Voice
-> Download Voice
-> Prepare Voice File
-> Route Voice File
-> Transcribe Voice
-> Apply Voice Transcript
-> Route Voice Transcript
-> Read Parser Context
-> Read Recent Memories
-> Apply Memory Context
-> OpenAI Parse Message
```

What happens:

1. The voice file is downloaded from Telegram. 2. The file is prepared for OpenAI transcription. 3. OpenAI transcribes the audio. 4. The transcript replaces the original message text. 5. The normal text parsing path continues.

If transcription fails, the workflow sends a helpful Telegram reply asking the user to resend or type the message.

9. Budget Commands

Budget commands are parsed directly in `Normalize Telegram Input`.

Supported examples:

```
groceries 2k
food limit 300 weekly
set monthly dining cap 1500
entertainment 1500aed
budget food 2,000
list budgets
```

Flow for setting a budget:

```
Route Pending Command
-> Prepare Budget Upsert
-> Upsert Budget
-> Confirm Budget Command
```

Flow for listing budgets:

```
Route Pending Command
-> Read Budgets
```

```
-> Format Budget List
-> Confirm Budget Command
```

Rules:

- `k` means thousand, so `2k` becomes `2000`
- Category text is preserved after cleanup
- `weekly` or `monthly` can be included
- Default period is monthly

10. Budget Warning After Expense Save

After saving an expense, the workflow checks the saved category's monthly budget.

Flow:

```
Append Expense
-> Read Budget After Expense
-> Read Monthly Spend After Expense
-> Append Budget Warning
-> Confirm Expense
```

If monthly spend is:

- 80 percent or more: append a warning
- 100 percent or more: append an over-budget warning

The expense is still saved. The warning is informational only.

11. Memory System

The assistant can remember preferences.

Examples:

```
remember Costa is Food
show memories
forget Costa
```

Memory creation flow:

```
Route Pending Command
-> Prepare Memory Embedding
-> Create Memory Embedding
-> Prepare Create Memory
-> Create Memory
-> Confirm Memory Command
```

Memory recall flow:

```
Route Pending Command
-> Read Memories for Recall
```



```
-> Prepare Memory Recall
-> Confirm Memory Command
```

Parser context flow:

```
Read Parser Context
-> Read Recent Memories
-> Apply Memory Context
-> OpenAI Parse Message
```

Apply Memory Context adds useful hints to the OpenAI system prompt, such as:

- Default currency
- Merchant category defaults
- Card aliases
- Recent memories

This helps OpenAI parse messages more consistently without hard-coding every behavior.

12. Task and Reminder Path

Task creation flow:

```
Route Intent
-> Prepare Task Row
-> Append Task
-> Confirm Task
```

Task completion flow:

```
Route Intent
-> Read Tasks for Done
-> Find Task to Complete
-> Route Task Match
-> Update Completed Task
-> Confirm Done
```

If a task cannot be matched clearly, the workflow replies with a useful message instead of updating the wrong task.

13. Undo Last Expense

Example:

```
undo
```

Flow:

```
Route Pending Command
-> Read Last Expense for Undo
-> Prepare Undo Expense
```

```
-> Route Undo Expense
-> Delete Last Expense
-> Confirm Undo Expense
```

The workflow reads the latest expense for the same chat/user context and deletes it if safe.

14. Supabase Tables and Views

The exact table names should be preserved because the workflow depends on them.

Important storage areas:

- `assistant_expenses`
- `assistant_tasks`
- `assistant_pending_actions`
- `assistant_memories`
- `assistant_budgets`
- `assistant_processed_telegram_updates`

The processed-update table exists to prevent duplicate work when Telegram retries a webhook.

15. How to Maintain This Workflow

Rule 1: Keep Node IDs Stable

n8n workflows are JSON files. Nodes have names and IDs. When possible, preserve existing IDs so imports do not create unnecessary churn.

Rule 2: Do Not Change Working Endpoints Casually

Be careful with:

- Supabase REST URLs
- Supabase table/view names
- Credential IDs
- Environment variable names
- Telegram webhook paths

Small changes here can break production behavior.

Rule 3: Update the Script, Then Regenerate the Workflow

Most hardening changes should go into:

```
scripts/harden-active-inbox-workflow.mjs
```

Then regenerate:

```
node scripts\harden-active-inbox-workflow.mjs
```

This keeps the exported workflow files reproducible.

Rule 4: Test Before Importing

Useful checks:

```
npm test -- --run
node -e "JSON.parse(require('fs').readFileSync('workflows/current-inbox-after-budget-commands.json', 'utf8')); console.log('workflow json ok')"
```

Rule 5: Import, Publish, Restart, Verify

Typical deployment commands:

```
n8n import:workflow --input=workflows\telegram-assistant-inbox-budget-commands.json
n8n publish:workflow --id=telegram-personal-assistant-inbox-budget-commands
.\start-n8n-with-ngrok.ps1
```

Then verify:

- Workflow is active in n8n
- Telegram webhook info is healthy
- Telegram test message receives a reply
- A real photo/voice/text test works if that path changed

16. How to Debug It

Use this process:

1. Find the latest n8n execution.
2. Identify the failed node.
3. Inspect that node's input and output.
4. Trace backward until you find where the data first became wrong.
5. Fix that source node.
6. Add a regression test if possible.
7. Import/publish/restart and run a live smoke test.

Common failure examples:

- No Telegram reply: check n8n execution status first.
- Duplicate expense: check Telegram update dedupe.
- Photo fails: inspect Prepare Photo Image and Apply Memory Context .
- Voice fails: inspect Download Voice , Prepare Voice File , and Transcribe Voice .
- Budget command misroutes: inspect budget parsing in Normalize Telegram Input .
- Confirmation affects the wrong item: inspect pending-action query filters.

17. How to Explain This Project in an Interview

A clear explanation:

```
I built an n8n-based Telegram personal assistant. Telegram sends messages to an n8n webhook. The workflow normalizes each update, deduplicates Telegram retries using Supabase, routes direct commands like budgets
```

and confirmations, and uses OpenAI for unstructured text, voice transcripts, and receipt image parsing. Structured results are validated and saved to Supabase. The bot replies back in Telegram with confirmations, clarification prompts, or budget warnings. I version the workflow JSON in Git, regenerate exports from scripts, run tests, import and publish to n8n, restart the local service, and verify webhook health plus live Telegram smoke tests.

If they ask about reliability:

I added Telegram update deduplication, retry settings on Telegram replies, scoped pending confirmations by chat ID and expiry, and regression tests that validate critical workflow wiring.

If they ask about debugging:

I debug by inspecting n8n execution data node by node. For example, a receipt-photo issue was caused by a later memory-context node overwriting the prepared base64 image. I traced the image payload through the workflow, found the first mutation that reintroduced the invalid value, fixed that node, added a test, redeployed, and confirmed with a live Telegram photo.

18. Mental Model

Think of the workflow as a small backend application drawn as boxes:

Controller: Telegram Inbox + Normalize Telegram Input
Middleware: Dedupe + Callback Ack + Memory Context
Router: Route Pending Command + Route Voice + Route Intent
Services: OpenAI parsing/transcription + Supabase REST calls
Database: Supabase assistant tables/views
Response layer: Telegram confirmation nodes

That is the same architecture you would use in code. n8n just makes the control flow visual.